
```

# Remove unnecessary packages (eg SPlit and Virtual Twins and psych)
# requiredPackages <-
# c('grf', 'policytree', 'data.table', 'randomForest', 'survival', 'SPlit', 'plyr',
# 'cubature', 'formatR', 'kableExtra', 'knitr', 'ggplot2', 'gridExtra', 'aVirtualTwins', 'psych')

requiredPackages <- c("grf", "policytree", "data.table", "randomForest", "survival",
  "plyr", "cubature", "knitr")

ipak <- function(pkg) {
  new.pkg <- pkg[!(pkg %in% installed.packages()[, "Package"])]
  if (length(new.pkg) > 0) {
    install.packages(new.pkg, dependencies = TRUE, quietly = TRUE, verbose = FALSE,
      logical.return = FALSE)
    sapply(pkg, require, character.only = TRUE, quietly = TRUE)
  }
}

ipak(requiredPackages)

rm(list = ls())

# Use common code source codepath<-c('/media/larryleon/My
# Projects/GitHub/Forest-Search/R/')
# source(paste0(codepath, 'source_forestsearch_v0.R'))
# source_fs_functions(file_loc=codepath)

# For GRF, only need the following source files
source("/media/larryleon/My Projects/GitHub/Forest-Search/R/sim_aft_gbsg-mod4_v0.R")
source("/media/larryleon/My Projects/GitHub/Forest-Search/R/grf_functions_v0.R")
source("/media/larryleon/My Projects/GitHub/Forest-Search/R/plotting_functions_v0.R")

# Data example output path outpath<-c('/media/larryleon/My Projects/Current
# Projects/subgroup identification and
# analysis/applications/simulated_ex1/output/')
library(knitr)
library(cubature)
library(randomForest)
library(survival)
library(grf)
library(policytree)
library(data.table)
library(plyr)

maxFollow <- 84
cens.type <- "weibull"
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# GRF criteria
dmin.grf <- 12 # For GRF delta --> dmin.grf/2 is benefit in favor of control
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

# Minimum # for subgroup H
n.min <- 60

```

```

# Simulated outcome names
outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

# m1 -censoring
muC.adj <- log(1.5)
# Here we add muC.adj to the Weibull intercept to adjust the degree of
# censoring Without adjustment muC.adj=0.0, then % censoring will reproduce
# the dataset This value results in ~46% censoring m0-censoring (original data)
# muC.adj<-0.0

# k.treat modifies the strength of the treatment effect of the Weibull model
# E.g., 0.90 reduces the beta value by 10%
k.treat <- 0.9
# k.z3 modifies the strength of Z3 which is the menopause factor that is
# involved in the interaction governing the subgroup generation Have not used
# this modification ... set to 1.0
k.z3 <- 1

# The interaction is between Z1 and Z3 where Z3=meno is a binary factor The %
# in subgroup H is  $I(Z1 \leq q) * I(Z3=1)$  where q is a quantile of Z1 So q
# determines the size since the proportion via Z3 is fixed in the dataset We
# let q=z1_frac with default 0.25

# NOTE that the largest proportion in H is ~40% and thus the complement lowest
# is 60% Would need to replace Z3 with another variable to allow for increasing
# the H

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

N <- 700

if (is.null(pH_super)) {
  # pH_check<-with(gbbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
}

```

```

    if (abs(rel_error) >= 0.1)
      stop("pH_super approximation relative error exceeds 10%")
    z1_frac <- z1_q
    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Estimation on log-scale but converted to HR (per coxph())
est.loghr <- TRUE
est.scale <- "hr"

# To output the file to specific location for accessing dgm later Turing off
# here
out.loc <- NULL
model.index <- NULL
file.index <- NULL

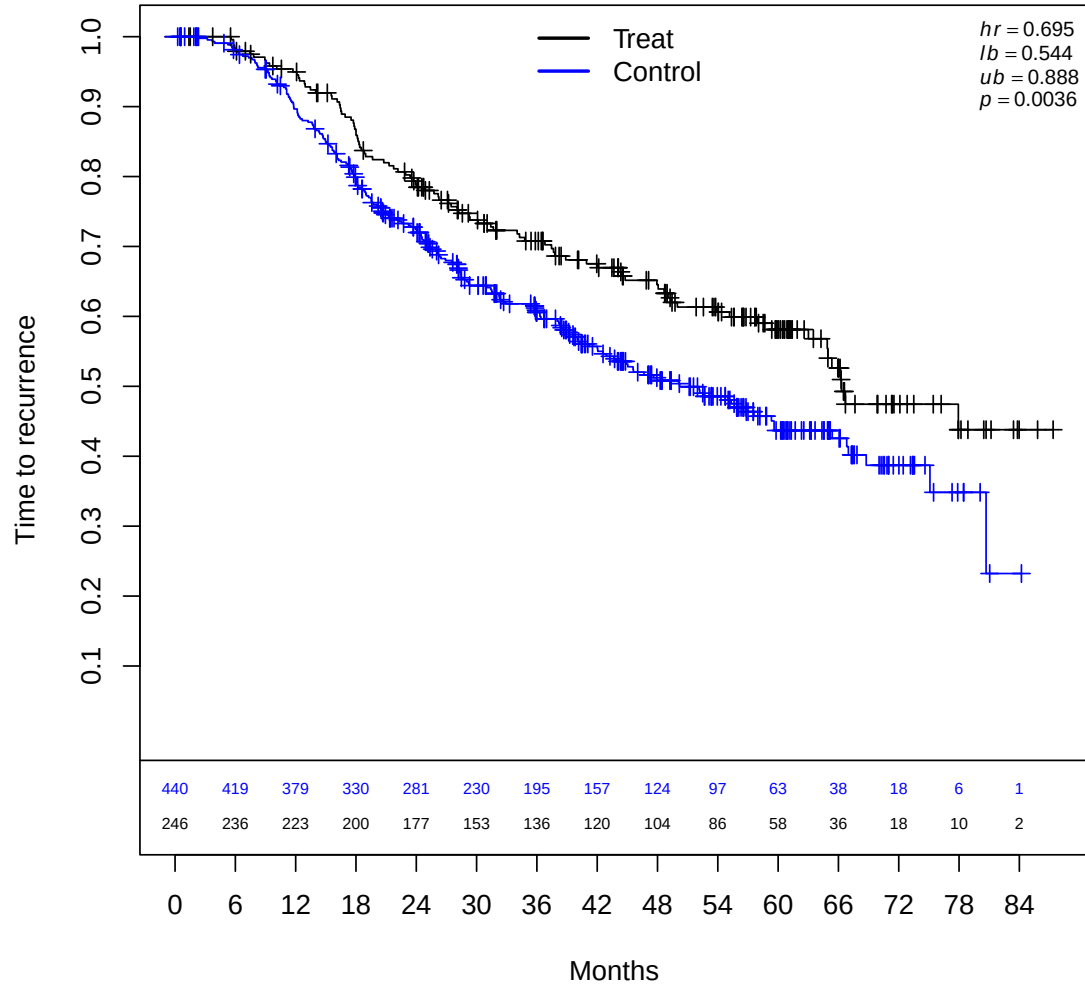
# mod.harm<-'null' return NO subgroup
mod.harm <- "null"

# hrh.target is the desired (marginal) hazard ratio for the subgroup set to
# NULL here since no subgroup (Though value doesn't matter as mod.harm will
# override)
hrH.target <- NULL

# details=TRUE will return KM plot of observed gbsg data

this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, model.index = model.index,
  hrH.target = hrH.target, cens.type = cens.type, out.loc = out.loc, file.index = file.index,
  details = TRUE)

```



```
## Super-population empirical harm and non-harm hazard ratios= NA 0.701027
## Causal HR (empirical ITT)= 0.701027
```

```
# Creates 'dgm' list containing
print(names(this.dgm$dgm))
```

```
## [1] "df.super_rand" "sigC"          "hr.H.true"      "hr.Hc.true"
## [5] "cens.type"     "mu"            "hr.causal"     "b.true"
## [9] "sig"          "muC"           "covs.analysis" "fs.harm.true"
## [13] "vt.harm.true" "grf.harm.true" "model"
```

```
# df.super_rand is the 'super population' of N=5000
print(names(this.dgm$dgm$df.super_rand))
```

```
## [1] "pid"      "age"      "meno"     "size"
## [5] "grade"    "nodes"    "pgr"      "er"
## [9] "hormon"   "rfstime"  "status"   "v7"
## [13] "v6"       "v5"       "v4"       "v3"
## [17] "v2"       "v1"       "flag.harm" "zh"
## [21] "z5"       "z3"       "z4"       "z2"
```

```

## [25] "z1"          "treat"        "event"        "id"
## [29] "y"           "hlin.conf.1"  "hlin.conf.0"  "lin1.conf"
## [33] "lin0.conf"    "lin.conf.true" "Ts"           "es"
## [37] "h0.potential" "h1.potential" "linC0.conf"   "linC1.conf"
## [41] "hlin.ratio"

# The original covariates and outcomes are returned ('super populaton'
# versions) h0.potential is the subjects' potential hazard for Control per
# Weibull model h1.potential is the subjects' potential hazard for Treatment
# per Weibull model

# The underlying marginal hazard ratio (average across potentials) for H is
print(this.dgm$dgm$hr.H.true)

## [1] NA

# Here it is NA since this is the null of NO subgroup So the complement and ITT
# are the same
print(this.dgm$dgm$hr.Hc.true)

##      treat
## 0.701027

# The ITT is 'hr.causal'
print(this.dgm$dgm$hr.causal)

##      treat
## 0.701027

# Now, in the simulations for GRF we output the identification of the H
# subgroup (and thus complement) that aligns with the implementation of GRF
# This is 'grf.harm.true'
print(this.dgm$dgm$grf.harm.true)

## NULL

# Is NULL here since under the null model (No subgroup)

##### For subgroup generation

# hrh.target is the desired (marginal) hazard ratio for the subgroup
hrH.target <- 2.5
mod.harm <- "alt"

this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, model.index = model.index,
  hrH.target = hrH.target, cens.type = cens.type, out.loc = out.loc, file.index = file.index,
  details = FALSE)

# The underlying marginal hazard ratio (average across potentials) for H is
print(this.dgm$dgm$hr.H.true)

##      treat
## 2.499996

# The complement
print(this.dgm$dgm$hr.Hc.true)

##      treat
## 0.6466405

```

```

# ITT
print(this.dgm$dgm$hr.causal)

##      treat
## 0.7125876

# Subgroup identification for GRF
print(this.dgm$dgm$grf.harm.true)

## [1] "v1=1" "v3=1"

# NOTE that as H effects (hrH.target) varies, the ITT effect will also vary BUT
# the complement effect is the same

# For example, changing hrH.target to 1.0
hrH.target <- 1
mod.harm <- "alt"
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, model.index = model.index,
  hrH.target = hrH.target, cens.type = cens.type, out.loc = out.loc, file.index = file.index,
  details = FALSE)
# The underlying marginal hazard ratio (average across potentials) for H is
print(this.dgm$dgm$hr.H.true)

##      treat
## 0.9999989

# The complement
print(this.dgm$dgm$hr.Hc.true)

##      treat
## 0.6466405

# ITT
print(this.dgm$dgm$hr.causal)

##      treat
## 0.6754515

# Subgroup identification for GRF
print(this.dgm$dgm$grf.harm.true)

## [1] "v1=1" "v3=1"

# Changing hrH.target to 3.0
hrH.target <- 3
mod.harm <- "alt"
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, model.index = model.index,
  hrH.target = hrH.target, cens.type = cens.type, out.loc = out.loc, file.index = file.index,
  details = FALSE)
# The underlying marginal hazard ratio (average across potentials) for H is
print(this.dgm$dgm$hr.H.true)

##      treat
## 3.000004

# The complement
print(this.dgm$dgm$hr.Hc.true)

##      treat
## 0.6466405

```

```
# ITT
print(this.dgm$dgm$hr.causal)
```

```
##      treat
## 0.7169919
```

```
# Subgroup identification for GRF
print(this.dgm$dgm$grf.harm.true)
```

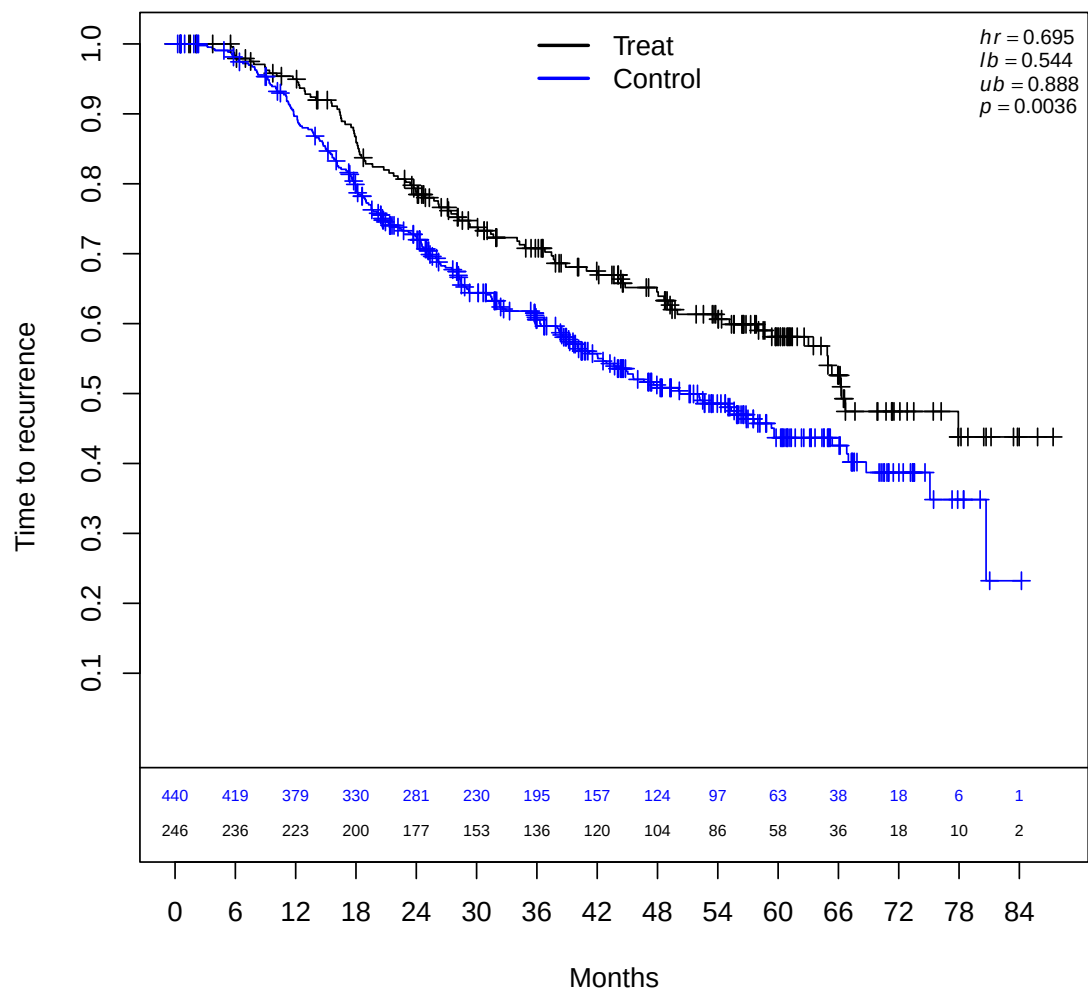
```
## [1] "v1=1" "v3=1"
```

```
# Simulate with hrH=2.5 pH ~ 13% (12.5)
```

```
hrH.target <- 2.5
```

```
mod.harm <- "alt"
```

```
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, model.index = model.index,
  hrH.target = hrH.target, cens.type = cens.type, out.loc = out.loc, file.index = file.index,
  details = TRUE)
```



```

## Super-population empirical harm and non-harm hazard ratios= 2.499996 0.6466405
## Causal HR (empirical ITT)= 0.7125876

# The underlying marginal hazard ratio (average across potentials) for H is
print(this.dgm$dgm$hr.H.true)

##      treat
## 2.499996

# The complement
print(this.dgm$dgm$hr.Hc.true)

##      treat
## 0.6466405

# ITT
print(this.dgm$dgm$hr.causal)

##      treat
## 0.7125876

# Subgroup identification for GRF
print(this.dgm$dgm$grf.harm.true)

## [1] "v1=1" "v3=1"

# The data-generating model for the sims is dgm

dgm <- this.dgm$dgm
# The covariates for the analysis
confounders.name <- dgm$covs.analysis
print(confounders.name)

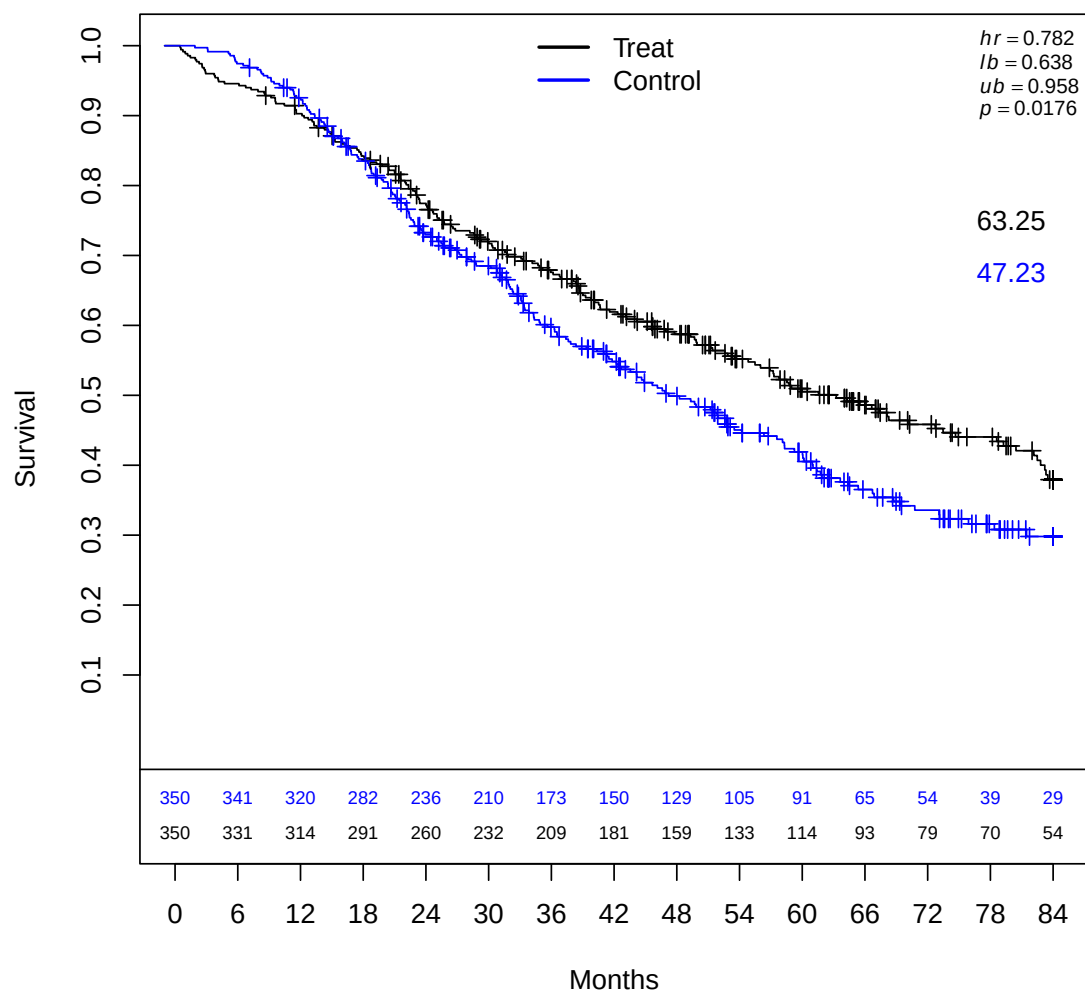
## [1] "v1" "v2" "v3" "v4" "v5" "v6" "v7"

# Note that the true model is the first 5 The last 2 are 'noise' covariates

# Randomly chose a simulation seed (based on the simulation number=sim) Note:
# sim<-1 returns NO estimated subgroup
sim <- 5
x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim,
  drawtreat = TRUE)

# KM curves for simulated data
plot_onesample(df = x, tte.name = "y.sim", event.name = "event.sim", treat.name = "treat",
  fix.rows = FALSE, byrisk = 6, show.med = TRUE)

```



```
# Returns simulated data (event.sim,y.sim) with covariates from the observed
# data flag.harm indicates whether the subject is in the H subgroup
print(table(x$flag.harm))

##
##    0    1
## 608  92

# The covariates and subgroups are random

# prop H summaries (%'s and sizes)
sizeH_true <- with(x, sum(flag.harm))
propH_true <- with(x, mean(flag.harm))
sizeHc_true <- with(x, sum(!flag.harm))
propHc_true <- with(x, mean(!flag.harm))
dfHpop <- data.table(sim, sizeH_true, propH_true, sizeHc_true, propHc_true)

print(dfHpop)

##    sim sizeH_true propH_true sizeHc_true propHc_true
```

```
## 1:      5          92  0.1314286          608   0.8685714

# GRF analysis with frac.tau=0.6 This uses 60% of the maximum truncation value
# for RMST That is, if t1 (t0) is max event time for treatment (control) Then
# RMST uses tau = 0.6*min(t0,t1)

ansGRF <- NULL
analysis <- "GRF"

showdetails <- TRUE

grf.est <- try(grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = frac.tau, details = showdetails),
  TRUE)

## tau= 48.87494
##      leaf.node control.mean control.size control.se treated.mean treated.size
## 1           2    -5.839009   391.000000   1.478196    5.839009   391.000000
## 2           3     4.382296   309.000000   1.540356   -4.382296   309.000000
## 3           2    -2.979152   490.000000   1.238637    2.979152   490.000000
## 4           4   -10.003601   118.000000   2.648085   10.003601   118.000000
## 5           5    18.600939    92.000000   2.804837   -18.600939    92.000000
## 11          5   -10.003601   118.000000   2.648085   10.003601   118.000000
## 21          8    -5.819074   239.000000   1.911168    5.819074   239.000000
## 51         11    -1.450641   107.000000   2.008444    1.450641   107.000000
## 6          12    -6.929135    63.000000   3.634465    6.929135    63.000000
## 7          13    20.627202    83.000000   2.881148   -20.627202    83.000000
##      treated.se      diff depth
## 1     1.478196 -11.678019     1
## 2     1.540356  8.764592     1
## 3     1.238637 -5.958305     2
## 4     2.648085 -20.007202     2
## 5     2.804837  37.201879     2
## 11     2.648085 -20.007202     3
## 21     1.911168 -11.638149     3
## 51     2.008444 -2.901282     3
## 6      3.634465 -13.858270     3
## 7      2.881148  41.254403     3
##      leaf.node control.mean control.size control.se treated.mean treated.size
## 7           13    20.627202    83.000000   2.881148   -20.627202    83.000000
##      treated.se      diff depth
## 7      2.881148  41.2544     3

# GRF applied to trees of depths 1, 2, and 3 tree 1 is depth 1 (etc)

print(names(grf.est))

## [1] "data"      "grf.gsub"   "sg.harm.id" "harm.any"   "tree"
## [6] "tau.rmst"  "tree1"     "tree2"     "tree3"

# 'tree' corresponds to the selected H subgroup

# Plotting does not work in knitr But these will show the forest trees

# plot(grf.est$tree1) plot(grf.est$tree2) plot(grf.est$tree3)
```

```

# The tree corresponding to selected subgroup plot(grf.est$tree)

# If a subgroup is selected (meets GRF criteria) then the subgroup id is output
print(grf.est$sg.harm.id)

## [1] "v1=2" "v2=2" "v3=2" "v4=2"

# If NO subgroup then sg.harm.id will be NULL

# Any subgroups with (causal) treatment effects in favor of control are stored
# here
print(grf.est$harm.any)

##   leaf.node control.mean control.size control.se treated.mean treated.size
## 2         3      4.382296   309.000000   1.540356      -4.382296   309.000000
## 5         5     18.600939    92.000000   2.804837     -18.600939    92.000000
## 7        13     20.627202    83.000000   2.881148     -20.627202    83.000000
##   treated.se      diff depth
## 2   1.540356   8.764592     1
## 5   2.804837  37.201879     2
## 7   2.881148  41.254403     3

# Next, output Cox model estimates

if (!inherits(grf.est, "try-error")) {

  ansGRF <- grf.estimates.out(df = x, grf.est = grf.est, dgm = dgm, cox.formula.sim = cox.formula,
    cox.formula.adj.sim = cox.formula.adj.sim, analysis = analysis, frac.tau = frac.tau)

  ansGRF <- cbind(dfHpop, ansGRF)

  print(names(ansGRF))

  # Plot the estimated subgroup

  # Note: grf.est$data is the analysis dataset with the GRF subgroup flag
  # attached 'treat.recommend=0' is the estimated H subgroup
  # 'treat.recommend=1' is the estimated H^c subgroup

  df0.grf <- subset(grf.est$data, treat.recommend == 0)
  df1.grf <- subset(grf.est$data, treat.recommend == 1)

  par(mfrow = c(1, 2))
  plot.subgroup(sub1 = df0.grf, sub1C = df1.grf, tte.name = "y.sim", event.name = "event.sim",
    treat.name = "treat", fix.rows = FALSE, byrisk = 9, legend.cex = 0.6, show.med = FALSE,
    subid = deparse(grf.est$sg.harm.id))

  # NOTE that the classification metrics naming is not standard npv is the %
  # of the estimated H^c (est(H^c), say) that are correctly classified
  # specificity is the % of the H^c population represented by est(H^c) ppv is
  # the % of the estimated H (est(H), say) that are correctly classified
  # sensitivity is the % of the H population represented by est(H)

  # The true size and 'estimated size' (size of estimated subgroup)
  print(ansGRF[, c("sizeH_true", "size.H")])

  # Complement

```

```

print(ansGRF[, c("sizeHc_true", "size.Hc")])

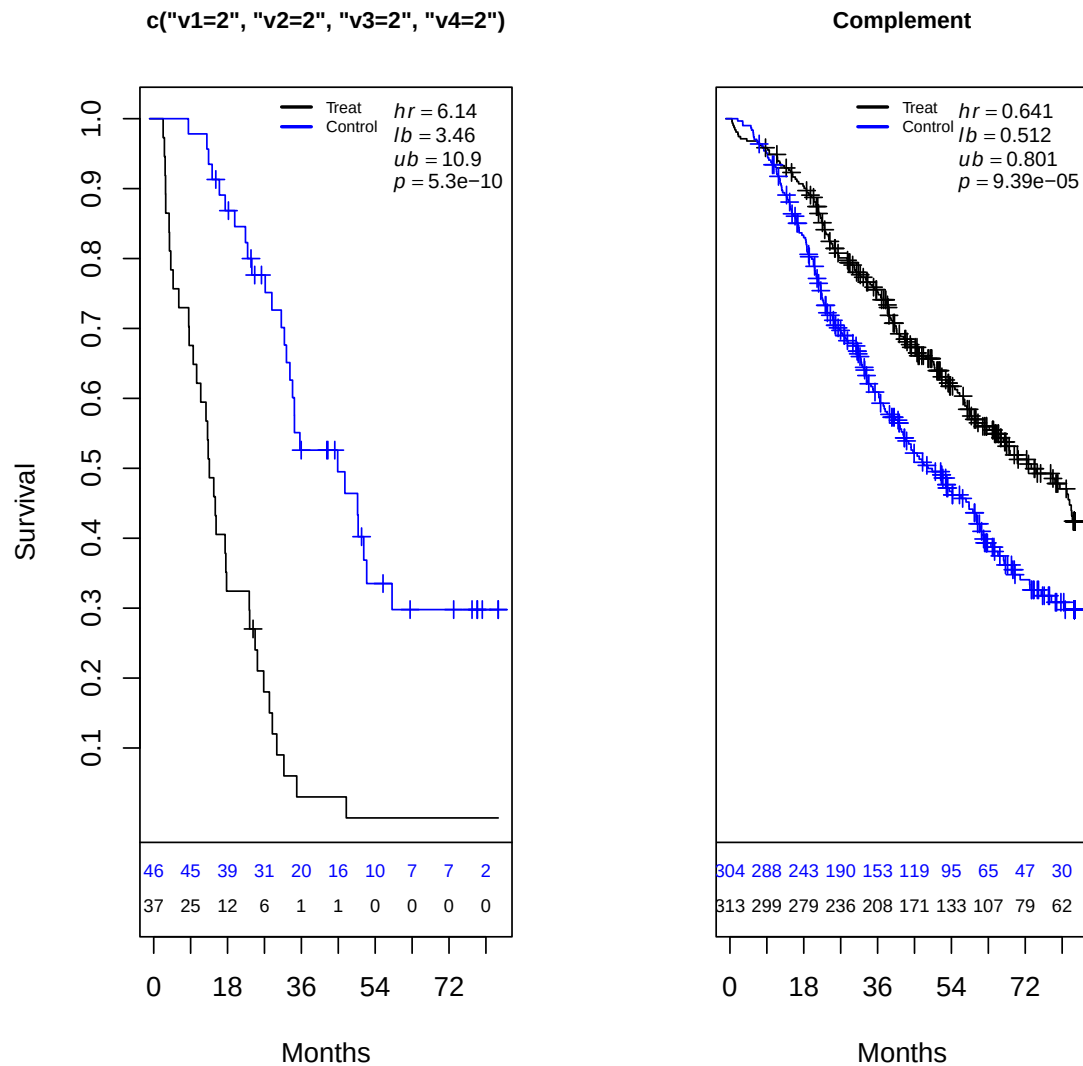
# Classification summaries
print(ansGRF[, c("npv", "specificity", "ppv", "sensitivity")])

# The Cox estimates and CIs for estimated subgroups These are all
# un-adjusted (raw) H true (est,lower,upper)
print(ansGRF[, c("hr.H.true", "l.H.true", "u.H.true")])
# GRF H estimate (est,lower,upper)
print(ansGRF[, c("hr.H.hat", "l.H.hat", "u.H.hat")])
# Complement H^c true (est,lower,upper)
print(ansGRF[, c("hr.Hc.true", "l.Hc.true", "u.Hc.true")])
# GRF H^c estimate (est,lower,upper)
print(ansGRF[, c("hr.Hc.hat", "l.Hc.hat", "u.Hc.hat")])

# ITT
print(ansGRF[, c("hr.itt", "l.itt", "u.itt")])
}

## [1] "sim"          "sizeH_true"   "propH_true"   "sizeHc_true"  "propHc_true"
## [6] "any.H"        "size.H"       "size.Hc"      "ppv"          "npv"
## [11] "specificity"  "sensitivity"  "found.1"      "found.2"      "found.both"
## [16] "found.al3"    "hr.H.true"    "hr.Hc.true"   "hr.H.hat"     "hr.Hc.hat"
## [21] "b1.H"         "b2.H"         "b1.Hc"        "b2.Hc"        "p.cens"
## [26] "analysis"     "taumax"       "hr.itt"        "l.itt"         "u.itt"
## [31] "hr.adj.itt"   "l.adj.itt"    "u.adj.itt"     "l.H.true"      "u.H.true"
## [36] "l.Hc.true"    "u.Hc.true"    "l.H.hat"       "u.H.hat"       "l.Hc.hat"
## [41] "u.Hc.hat"

```



```
## sizeH_true size.H
## 1: 92 83
## sizeHc_true size.Hc
## 1: 608 617
## npv specificity ppv sensitivity
## 1: 1 0.9854133 0.9021739 1
## hr.H.true l.H.true u.H.true
## 1: 4.75023 2.841556 7.940962
## hr.H.hat l.H.hat u.H.hat
## 1: 6.136099 3.461193 10.87825
## hr.Hc.true l.Hc.true u.Hc.true
## 1: 0.6259481 0.499205 0.78487
## hr.Hc.hat l.Hc.hat u.Hc.hat
## 1: 0.6407118 0.5124452 0.8010839
## hr.itt l.itt u.itt
## 1: 0.7819166 0.6382393 0.9579379
```

*# If GRF is not 'estimable' then consider NO subgroup found Consider grf.est =
NULL*

```
if (inherits(grf.est, "try-error")) {  
  ansGRF <- grf.estimate.out(df = x, grf.est = NULL, dgm = dgm, cox.formula.sim = cox.formula.sim,  
    cox.formula.adj.sim = cox.formula.adj.sim, analysis = analysis, frac.tau = 1)  
  ansGRF <- cbind(dfHpop, ansGRF)  
  if (round(c(ansGRF$hr.Hc.hat - ansGRF$hr.itt), 6) != 0)  
    stop("Cox hr for H-complement NOT equal to ITT (GRF)")  
}
```